

```

const puppeteer = require('puppeteer-extra');
const StealthPlugin = require('puppeteer-extra-plugin-stealth');
const axios = require('axios');

puppeteer.use(StealthPlugin());

// =====
// CONFIGURATION
// =====

const ASSET_1_SHEET_API =
"https://script.google.com/macros/s/AKfycbzP0gBXxSyEmoq7bMCXl61ur7a95hn9JaeZCX7G99d2KzV5o6TCBvwFaqqmSv8mTDY9hw/exec";

const ASSET_2_SHEET_API =
"https://script.google.com/macros/s/AKfycbylZa7JbhWa3DDQlBLsQ8QbawB-2Z1dbZV2AzcABxxGqGLnEqq9DxwBMLg22ngDqPEA/exec";

const MODERN_USER_AGENT = "Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/122.0.0.0 Safari/537.36";

/**
 * Resilient page navigation pattern with built-in retry logic.
 */
async function safeNavigateWithRetry(page, url, options, retries = 3) {
  for (let i = 0; i < retries; i++) {
    try {
      await page.goto(url, options);
      return;
    } catch (error) {
      if (i === retries - 1) throw new Error(`[FATAL DESTINATION FAILURE] Node dropped after ${retries} attempts: ${error.message}`);
    }
  }
}

```

```
    console.log(` ⚠ Network retry trigger active (${i + 1}/${retries}). Re-attempting connection...`);
```

```
    await new Promise(resolve => setTimeout(resolve, 3000));
```

```
  }
```

```
}
```

```
}
```

```
/**
```

```
 * Dismisses Google's cookie/consent interstitial if it appears.
```

```
 * This is the #1 cause of "empty results" when running headless on a server.
```

```
 */
```

```
async function dismissConsentIfPresent(page) {
```

```
  try {
```

```
    const consentButtonSelectors = [
```

```
      'button[aria-label="Accept all"]',
```

```
      'form[action*="consent"] button',
```

```
      'button:has-text("Accept all")',
```

```
    ];
```

```
    for (const sel of consentButtonSelectors) {
```

```
      const btn = await page.$(sel).catch(() => null);
```

```
      if (btn) {
```

```
        await btn.click();
```

```
        await new Promise(r => setTimeout(r, 1500));
```

```
        console.log('✅ Dismissed consent interstitial.');
```

```
        return;
```

```
      }
```

```
    }
```

```
    // Fallback: click any button whose text looks like an accept action
```

```
    const clicked = await page.evaluate(() => {
```

```

const buttons = Array.from(document.querySelectorAll('button'));

const target = buttons.find(b => /accept all|i agree/i.test(b.textContent || ''));

if (target) { target.click(); return true; }

return false;

});

if (clicked) {

  await new Promise(r => setTimeout(r, 1500));

  console.log('✅ Dismissed consent interstitial (text match).');

}

} catch (e) {

  // Non-fatal — page may simply not have shown a consent wall

}

}

/**
 * Fallback contact lookup against a public business directory (no login required).
 * Note: directory site markup changes over time just like Maps — if this stops
 * matching, inspect the live DOM and update the selectors below.
 */

async function searchDirectoryFallback(browser, businessName, locationHint) {

  const page = await browser.newPage();

  try {

    await page.setUserAgent(MODERN_USER_AGENT);

    const searchUrl =
`https://www.yellowpages.com/search?search_terms=${encodeURIComponent(businessName)}&geo_location_terms=${encodeURIComponent(locationHint || '')}`;

    await safeNavigateWithRetry(page, searchUrl, { waitUntil: 'domcontentloaded',
timeout: 15000 });

    const detailHref = await page.evaluate(() => {

```

```

    const result = document.querySelector('.result .business-name');
    return result ? result.getAttribute('href') : null;
  });

  if (!detailHref) return null;

  const detailUrl = detailHref.startsWith('http') ? detailHref :
`https://www.yellowpages.com${detailHref}`;

  await safeNavigateWithRetry(page, detailUrl, { waitUntil: 'domcontentloaded', timeout:
15000 });

  const email = await page.evaluate(() => {
    const mailtoAnchor = document.querySelector('a[href^="mailto:"]');
    if (mailtoAnchor) return mailtoAnchor.getAttribute('href').replace(/mailto:/i,
'').split('?')[0].trim();

    const emailRegex = /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g;
    const matches = document.body.innerText.match(emailRegex);
    return matches ? matches[0] : null;
  });

  return email;
} catch (e) {
  return null;
} finally {
  await page.close();
}
}

/**

```

- * Visits a place's own Maps detail page to resolve the canonical GBP URL
- * and the official website link Google itself associates with the listing.
- * List-card values for these are frequently missing or session-scoped —
- * the detail page is the authoritative source for both.
- */

```

async function getPlaceDetails(browser, gbpMapUrl) {
  const page = await browser.newPage();

  try {
    await page.setUserAgent(MODERN_USER_AGENT);

    const fullUrl = gbpMapUrl.startsWith('http') ? gbpMapUrl :
`https://www.google.com${gbpMapUrl}`;

    await safeNavigateWithRetry(page, fullUrl, { waitUntil: 'domcontentloaded', timeout:
20000 });

    // Give the detail panel a moment to render before reading it
    await page.waitForSelector('div[role="main"]', { timeout: 8000 }).catch(() => null);

    await new Promise(r => setTimeout(r, 1200)); // settle buffer for late-rendering action
buttons

    const details = await page.evaluate(() => {
      // data-item-id="authority" is fragile and drifts with Google's markup updates.
      // The reliable signal is simpler: an <a> whose href points to a real external
      // domain (not a google.com/maps/gstatic link). Try the known attribute first,
      // then fall back to scanning all links on the page.
      const isExternalWebsiteLink = (a) => {
        const href = a.getAttribute('href') || '';
        if (!href.startsWith('http')) return false;
        if (href.includes('google.com') || href.includes('gstatic.com')) return false;
        return true;
      };
    });
  }
}

```

```

let websiteEl = document.querySelector('a[data-item-id="authority"]');
if (!websiteEl || !isExternalWebsiteLink(websiteEl)) {
    const candidates = Array.from(document.querySelectorAll('a[href^="http"]'));
    websiteEl = candidates.find(a => {
        if (!isExternalWebsiteLink(a)) return false;
        const label = (a.getAttribute('aria-label') || a.textContent || '').toLowerCase();
        return label.includes('website') || label.includes('site');
    }) || candidates.find(isExternalWebsiteLink) || null;
}

const phoneEl = document.querySelector('button[data-item-id^="phone"]');
const phoneLabel = phoneEl ? (phoneEl.getAttribute('aria-label') ||
phoneEl.textContent || '').replace(/^Phone:\s*/i, '').trim() : null;

return {
    website: websiteEl ? websiteEl.getAttribute('href') : null,
    phone: phoneLabel || null,
};
});

// page.url() after navigation is the canonical, stable GBP URL —
// unlike the search-result href, it won't expire with the session.
return { gbp_url: page.url(), website: details.website, phone: details.phone };
} catch (e) {
    return { gbp_url: gbpMapUrl, website: null, phone: null };
} finally {
    await page.close();
}
}

```

```
/**
```

- * Extracts a contact email from the currently loaded page. Checks, in order:
- * direct mailto links, Cloudflare-obfuscated emails (a very common anti-scraping
- * measure that a plain mailto/regex check silently misses), and finally a
- * regex sweep of the full HTML rather than just visible text.

```
*
```

- * The Cloudflare decoder is a simple, publicly documented single-byte XOR
- * cipher — it's undoing obfuscation of data the site already serves to
- * every visitor's browser, not bypassing any actual security control.

```
*/
```

```
async function extractEmailFromCurrentPage(page) {
```

```
  return await page.evaluate(() => {
```

```
    const decodeCfEmail = (encoded) => {
```

```
      let email = "";
```

```
      const key = parseInt(encoded.substr(0, 2), 16);
```

```
      for (let i = 2; i < encoded.length; i += 2) {
```

```
        const charCode = parseInt(encoded.substr(i, 2), 16) ^ key;
```

```
        email += String.fromCharCode(charCode);
```

```
      }
```

```
      return email;
```

```
    };
```

```
// 1. Direct mailto anchors
```

```
const mailtoAnchors = document.querySelectorAll('a[href^="mailto:"]');
```

```
for (const anchor of mailtoAnchors) {
```

```
  const clean = anchor.getAttribute('href').replace(/mailto:/i, "").split('?')[0].trim();
```

```
  if (clean) return clean;
```

```
}
```

```

// 2. Cloudflare-obfuscated emails — extremely common on small
// business sites, and invisible to a plain mailto/regex check
const cfProtected = document.querySelectorAll('[data-cfemail]');
for (const el of cfProtected) {
  try {
    const decoded = decodeCfEmail(el.getAttribute('data-cfemail'));
    if (decoded && decoded.includes('@')) return decoded;
  } catch (e) { /* malformed hash, skip */ }
}

// 3. Regex across the full HTML (not just visible innerText) —
// catches emails sitting in footers, comments, or elements hidden
// by CSS that innerText would skip
const emailRegex = /[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/g;
const htmlMatches = document.documentElement.innerHTML.match(emailRegex);
if (htmlMatches) {
  const filtered = htmlMatches.filter(e =>
    !/\.(png|jpg|jpeg|gif|svg|webp)$/i.test(e) &&
    !e.includes('sentry') && !e.includes('example.com') && !e.includes('wixpress')
  );
  if (filtered.length > 0) return filtered[0];
}

return null;
});
}

/**

```


- * Tries the homepage first, then falls through common contact-page paths
- * if nothing is found there. Most small business sites keep the actual
- * email off the homepage entirely.
- */

```

async function findEmailOnWebsite(browser, websiteUrl) {
  const contactPage = await browser.newPage();

  try {
    await contactPage.setUserAgent(MODERN_USER_AGENT);

    // networkidle2 (not domcontentloaded) — footers with contact info
    // are frequently injected by JS after the initial DOM parse
    await safeNavigateWithRetry(contactPage, websiteUrl, { waitUntil: 'networkidle2',
    timeout: 20000 });

    await new Promise(r => setTimeout(r, 800));

    let email = await extractEmailFromCurrentPage(contactPage);
    if (email) return email;

    const base = websiteUrl.replace(/\/$/, '');
    const candidatePaths = ['/contact', '/contact-us', '/contactus', '/pages/contact',
    '/about/contact', '/contact.html'];

    for (const path of candidatePaths) {
      try {
        await safeNavigateWithRetry(contactPage, base + path, { waitUntil:
        'domcontentloaded', timeout: 12000 });

        await new Promise(r => setTimeout(r, 600));

        email = await extractEmailFromCurrentPage(contactPage);
        if (email) return email;
      } catch (e) {

```

```

        continue; // path likely 404s — try the next candidate
    }
}

return null;
} catch (e) {
    return null;
} finally {
    await contactPage.close();
}
}

async function scrapeGoogleMapsUnlimited(searchQuery, targetWebhookUrl) {
    console.log(`🚀 INITIALISING ENTERPRISE PUPPETEER CRAWL FOR: [${searchQuery}]`);

    // Used as the geo hint for the directory fallback, e.g. "Houston TX" from "HVAC
    Equipment Distributors in Houston TX"

    const locationHint = searchQuery.includes(' in ') ? searchQuery.split(' in ').pop() : '';

    console.log('🚀 Launching browser...');

    const browser = await puppeteer.launch({
        headless: "new", // "new" headless is far less likely to be flagged than the legacy
        true/false mode

        args: [
            '--no-sandbox',
            '--disable-setuid-sandbox',
            '--disable-gpu',
            '--disable-web-security',
            '--disable-features=IsolateOrigins,site-per-process',
            '--disable-dev-shm-usage' // critical on VPS/Docker: default /dev/shm (often 64MB)
            causes

```

```

        // Chrome to hang or crash silently under memory pressure otherwise
    ]
});

console.log('✅ Browser launched successfully.');
```



```

try {
    const page = await browser.newPage();

    page.setDefaultNavigationTimeout(30000); // safety net in case any goto() call is ever
missing an explicit timeout

    await page.setUserAgent(MODERN_USER_AGENT);

    await page.setViewport({ width: 1280, height: 900 });

    // Force English UI so our class/text-based selectors stay predictable

    const searchUrl = "https://www.google.com/maps/search/" +
encodeURIComponent(searchQuery) + "?hl=en";

    await safeNavigateWithRetry(page, searchUrl, { waitUntil: 'networkidle2', timeout:
60000 });

    await dismissConsentIfPresent(page);

    // Wait explicitly for the results feed instead of assuming it's already there.
    // If this throws, we log diagnostics instead of silently returning nothing.

    try {
        await page.waitForSelector('div[role="feed"]', { timeout: 15000 });
    } catch (e) {
        const title = await page.title().catch(() => 'unknown');

        const url = page.url();

        console.log(`❌ Results feed never appeared. Page title: "${title}" | URL: ${url}`);

        console.log(` This usually means Google served a consent page, CAPTCHA, or "no
results" state.`);
    }
}

```

```
    await browser.close();  
    return;  
}
```

```
let previousCount = 0;  
let currentCount = 0;  
let sameCountCycles = 0;
```

```
while (sameCountCycles < 10) {  
  try {  
    await page.evaluate(() => {  
      const scrollPanel = document.querySelector('div[role="feed"]');  
      if (scrollPanel) scrollPanel.scrollTo(0, scrollPanel.scrollHeight);  
    });  
    await new Promise(resolve => setTimeout(resolve, 2500));  
  
    currentCount = await page.evaluate(() =>  
document.querySelectorAll('div.Nv2PK').length);  
  
    console.log(`⌚ Map List Density: Rendered ${currentCount} business nodes...`);  
  
    if (currentCount === previousCount) {  
      sameCountCycles++;  
    } else {  
      sameCountCycles = 0;  
    }  
  
    if (currentCount >= 50) break;  
    previousCount = currentCount;  
  } catch (scrollError) {
```

```

        console.log(` ⚠ Scroll loop stopped early: ${scrollError.message}`);
        break;
    }
}

// FIX: only require name + place link at the list-card level.
// The website link is frequently absent from the card and needs
// to be picked up from the detail panel instead — requiring it
// up front was silently discarding almost every listing.
const listingsData = await page.evaluate(() => {
    const records = [];
    const uniquePlaces = new Set();

    const cards = document.querySelectorAll('div.Nv2PK');

    cards.forEach(card => {
        const linkEl = card.querySelector('a[href*="/maps/place/"]');
        const nameEl = card.querySelector('.qBF1Pd') ||
card.querySelector('.fontHeadlineSmall');

        const isExternalWebsiteLink = (a) => {
            const href = a.getAttribute('href') || '';
            if (!href.startsWith('http')) return false;
            if (href.includes('google.com') || href.includes('gstatic.com')) return false;
            return true;
        };

        let websiteEl = card.querySelector('a[data-item-id="authority"]');
        if (!websiteEl || !isExternalWebsiteLink(websiteEl)) {
            const candidates = Array.from(card.querySelectorAll('a[href^="http"]'));

```

```

websiteEl = candidates.find(a => {
  if (!isExternalWebsiteLink(a)) return false;
  const label = (a.getAttribute('aria-label') || a.textContent || '').toLowerCase();
  return label.includes('website') || label.includes('site');
}) || candidates.find(isExternalWebsiteLink) || null;
}

const phoneEl = card.querySelector('button[data-item-id^="phone"]') ||
  card.querySelector('span.UsdlK');

if (linkEl && nameEl) {
  const nameText = nameEl.textContent.trim();
  const gbpMapUrl = linkEl.getAttribute('href');
  const siteUrl = websiteEl ? websiteEl.getAttribute('href') : null;
  const phoneText = phoneEl ? phoneEl.textContent.trim() : null;

  if (!uniquePlaces.has(nameText)) {
    uniquePlaces.add(nameText);
    records.push({ name: nameText, website: siteUrl, gbp_link: gbpMapUrl, phone:
phoneText });
  }
}

});

return records;
});

```

console.log(`\n  Extraction Complete. Isolated \${listingsData.length} business profiles
 (\${listingsData.filter(r => r.website).length} with a direct website link).`);

```

if (listingsData.length === 0) {

```

console.log(' ✖ Zero listings parsed even though the feed loaded — the card selector (div.Nv2PK) likely changed. Inspect the live DOM to confirm.');

}

for (let record of listingsData) {

 // Resolve the canonical GBP URL and official website from the place's

 // own detail page before deciding how to find a contact email.

 console.log(`\n🔑 Resolving place details for [\${record.name}]...`);

 const details = await getPlaceDetails(browser, record.gbp_link);

 record.gbp_link = details.gbp_url || record.gbp_link;

 record.website = record.website || details.website;

 record.phone = record.phone || details.phone;

 let targetEmail = null;

 let contactSource = null;

 if (record.website) {

 console.log(`\n🔑 Scanning website architecture for contact endpoints: \${record.website}`);

 try {

 targetEmail = await findEmailOnWebsite(browser, record.website);

 if (targetEmail) contactSource = 'website';

 } catch (pageError) {

 console.log(` ⚠ Navigation skipped for [\${record.name}] due to domain connection timeout error.`);

 }

 }

 if (!targetEmail) {

```
    console.log(`🔍 No email via website for [${record.name}] — checking public  
directory listing...`);
```

```
    const directoryEmail = await searchDirectoryFallback(browser, record.name,  
locationHint);
```

```
    if (directoryEmail) {
```

```
        targetEmail = directoryEmail;
```

```
        contactSource = 'directory';
```

```
        console.log(`✅ Directory fallback found an email for [${record.name}].`);
```

```
    }
```

```
    await new Promise(resolve => setTimeout(resolve, 1500)); // pacing between  
directory requests
```

```
}
```

```
if (!targetEmail && record.phone) {
```

```
    contactSource = 'maps-phone-only';
```

```
    console.log(`📞 No email found anywhere for [${record.name}] — falling back to  
phone from Maps listing.`);
```

```
}
```

```
if (!targetEmail && !record.phone) {
```

```
    console.log(`🚫 Skipping [${record.name}] — no email or phone found via  
website, directory, or Maps listing.`);
```

```
    continue;
```

```
}
```

```
const payload = {
```

```
    organization_name: record.name,
```

```
    operational_email: targetEmail ? targetEmail.toString().toLowerCase().trim() : null,
```

```
    phone: record.phone || null,
```

```
    contact_source: contactSource,
```

```
    website_url: record.website,
```



```

        gbp_url: record.gbp_link
    };

    try {
        const response = await axios.post(targetWebhookUrl, payload, { headers: {
'Content-Type': 'application/json' } });
        if (response.status === 200) {
            console.log(`✉ Pipeline Cleared: Lead [${record.name}] pushed to sheet
(source: ${contactSource}).`);
        }
        await new Promise(resolve => setTimeout(resolve, 2000));
    } catch (apiError) {
        console.log(`❌ Target connection drop or duplication skip for [${record.name}]`);
    }
}
} finally {
    await browser.close();
    console.log(`🚩 Outreach Sequence for [${searchQuery}] Concluded.\n`);
}
}

```

```

async function main() {
    // Watchdog: if the whole run somehow exceeds this, force an exit with a
    // clear message rather than leaving a blinking cursor with no explanation.
    // 20 minutes covers two full search batches with generous margin; tune as needed.
    const WATCHDOG_MS = 20 * 60 * 1000;
    const watchdog = setTimeout(() => {
        console.error(`\n🕒 WATCHDOG TRIGGERED: script exceeded ${WATCHDOG_MS /
60000} minutes without completing.`);
    });
}

```

```
    console.error(' This almost always means the browser process hung — check /dev/shm size,');
```

```
    console.error(' Chromium system dependencies, and outbound network access on this machine.');
```

```
    process.exit(1);
```

```
  }, WATCHDOG_MS);
```

```
  watchdog.unref(); // don't let the watchdog itself keep the process alive once real work finishes
```

```
const nichesAsset1 = [
```

```
  "Industrial Supply Distributors",
```

```
  "Wholesale Plumbing Supplies",
```

```
  "Electrical Wholesalers",
```

```
  "HVAC Equipment Distributors",
```

```
  "Logistics & Warehousing Companies"
```

```
];
```

```
const hubsAsset1 = ["Houston TX", "Chicago IL", "Atlanta GA", "Dallas TX", "Charlotte NC"];
```

```
const query1 = nichesAsset1[Math.floor(Math.random() * nichesAsset1.length)] + " in " + hubsAsset1[Math.floor(Math.random() * hubsAsset1.length)];
```

```
const nichesAsset2 = ["Independent Insurance Agencies", "Life Insurance Brokers", "Health Insurance Distributors", "Auto Insurance Agents"];
```

```
const hubsAsset2 = ["Dallas TX", "Miami FL", "Orlando FL", "Houston TX", "Atlanta GA"];
```

```
const query2 = nichesAsset2[Math.floor(Math.random() * nichesAsset2.length)] + " in " + hubsAsset2[Math.floor(Math.random() * hubsAsset2.length)];
```

```
await scrapeGoogleMapsUnlimited(query1, ASSET_1_SHEET_API);
```

```
await scrapeGoogleMapsUnlimited(query2, ASSET_2_SHEET_API);
```

```
clearTimeout(watchdog);
```

```
}
```

```
main();
```